

# CartServlet.java

```
package com.harisudhan.harisudhanmart.controller;

import com.harisudhan.harisudhanmart.dao.DAOFactory;
import com.harisudhan.harisudhanmart.exception.NotFoundException;
import com.harisudhan.harisudhanmart.exception.ValidationException;
import com.harisudhan.harisudhanmart.service.CartService;
import com.google.gson.JsonObject;
import java.io.IOException;
import java.sql.SQLException;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;

/** F4: cart add/update/remove and running total. AuthFilter guarantees a logged-in session. */
@WebServlet(urlPatterns = {"/api/v1/cart", "/api/v1/cart/add", "/api/v1/cart/update", "/api/v1/cart/remove"})
public class CartServlet extends BaseServlet {

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse resp) throws ServletException, IOException {
        long userId = currentUserId(req.getSession(false));
        DAOFactory factory = daoFactory();
        CartService service = new CartService(factory.cartDAO(), factory.productDAO());
        try {
            writeJson(resp, HttpServletResponse.SC_OK, service.viewCart(userId));
        } catch (SQLException e) {
            writeError(resp, HttpServletResponse.SC_INTERNAL_SERVER_ERROR, "DB_ERROR", "Could not load cart");
        }
    }

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse resp) throws ServletException, IOException {
        HttpSession session = req.getSession(false);
        long userId = currentUserId(session);
        DAOFactory factory = daoFactory();
        CartService service = new CartService(factory.cartDAO(), factory.productDAO());
        JsonObject body = com.google.gson.JsonParser.parseString(readBody(req)).getAsJsonObject();
        String path = req.getServletPath();

        try {
            if (path.endsWith("/add")) {
                long productId = body.get("productId").getAsLong();
                int quantity = body.get("quantity").getAsInt();
                writeJson(resp, HttpServletResponse.SC_CREATED, service.addItem(userId, productId, quantity));
            } else if (path.endsWith("/update")) {
                long cartItemId = body.get("cartItemId").getAsLong();
                int quantity = body.get("quantity").getAsInt();
                service.updateQuantity(userId, cartItemId, quantity);
                writeJson(resp, HttpServletResponse.SC_OK, null);
            } else if (path.endsWith("/remove")) {
                long cartItemId = body.get("cartItemId").getAsLong();
                service.removeItem(userId, cartItemId);
                writeJson(resp, HttpServletResponse.SC_OK, null);
            } else {
                writeError(resp, HttpServletResponse.SC_NOT_FOUND, "NOT_FOUND", "Unknown cart endpoint");
            }
        } catch (ValidationException e) {
            writeError(resp, HttpServletResponse.SC_BAD_REQUEST, e.getFieldErrorCode(), e.getMessage());
        } catch (NotFoundException e) {
            writeError(resp, HttpServletResponse.SC_NOT_FOUND, "NOT_FOUND", e.getMessage());
        } catch (SQLException e) {
            writeError(resp, HttpServletResponse.SC_INTERNAL_SERVER_ERROR, "DB_ERROR", "Cart update failed");
        }
    }
}
```

```
    }  
}  
  
private String readBody(HttpServletRequest req) throws IOException {  
    StringBuilder sb = new StringBuilder();  
    String line;  
    while ((line = req.getReader().readLine()) != null) {  
        sb.append(line);  
    }  
    return sb.length() == 0 ? "{}" : sb.toString();  
}  
}
```